

Air University

Department of Computer Science

Introduction to Parallel & Distributed Computing (CS426L)

Lab Report 04: Concurrency Issues

Student Name	Fahad Noor
Registration No.	232448
Section	BSCS-ev-7-C
Instructor	Muhammad Nauman Zafar Hashmi
Date	30 September 2026

Section 1: Environment Setup

Confirmed via `dotnet --version` that the .NET SDK carried over from Labs 02/03 is still available. Noted `Environment.ProcessorCount` for this machine (see Section 2, Task 1 results below). A working folder `PDC_Lab04` was created, containing four sub-projects: `RaceLab`, `RaceTrace`, `MutexFix`, and `CounterBench`.

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo>dotnet --version
10.0.401
```

Section 2: Task 1 Unsynchronized Counter (Shared-State Hazard)

Source file: `RaceLab/Program.cs`

```
using System;
using System.Threading;

class Program
{
    const int NumThreads = 4;
    const int IncrementsPerThread = 1_000_000;

    static long counter = 0; // shared state

    static void Worker()
    {
        for (int i = 0; i < IncrementsPerThread; i++)
        {
            counter++; // unsynchronized read-modify-write
        }
    }

    static void Main()
    {
        Thread[] threads = new Thread[NumThreads];

        for (int i = 0; i < NumThreads; i++)
        {
            threads[i] = new Thread(Worker);
            threads[i].Start();
        }

        for (int i = 0; i < NumThreads; i++)
        {
            threads[i].Join();
        }

        long expected = (long)NumThreads * IncrementsPerThread;
        Console.WriteLine($"Expected: {expected}");
        Console.WriteLine($"Actual: {counter}");
        Console.WriteLine($"Lost updates: {expected - counter}");
    }
}
```

Results (10 runs)

Run	Expected Value	Actual Value	Lost Updates
1	4,000,000	1,446,159	2,553,841
2	4,000,000	1,862,342	2,137,658
3	4,000,000	1,463,667	2,536,333
4	4,000,000	1,356,618	2,643,382
5	4,000,000	1,540,443	2,459,557
6	4,000,000	1,330,044	2,669,956

7	4,000,000	1,468,631	2,531,369
8	4,000,000	1,413,766	2,586,234
9	4,000,000	1,420,470	2,579,530
10	4,000,000	1,704,134	2,295,866

Number of logical cores on this machine (Environment.ProcessorCount): 8

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceLab>dotnet build
Restore complete (1.0s)
  RaceLab net10.0 succeeded (6.1s) → bin\Debug\net10.0\RaceLab.dll

Build succeeded in 8.1s

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceLab>dotnet run --no-build
Expected: 4000000
Actual: 1446159
Lost updates: 2553841

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceLab>dotnet run --no-build
Expected: 4000000
Actual: 1862342
Lost updates: 2137658

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceLab>dotnet run --no-build
Expected: 4000000
Actual: 1463667
Lost updates: 2536333

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceLab>dotnet run --no-build
Expected: 4000000
Actual: 1356618
Lost updates: 2643382

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceLab>dotnet run --no-build
Expected: 4000000
Actual: 1540443
Lost updates: 2459557
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceLab>dotnet run --no-build
Expected: 4000000
Actual: 1330044
Lost updates: 2669956

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceLab>dotnet run --no-build
Expected: 4000000
Actual: 1468631
Lost updates: 2531369

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceLab>dotnet run --no-build
Expected: 4000000
Actual: 1413766
Lost updates: 2586234

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceLab>dotnet run --no-build
Expected: 4000000
Actual: 1420470
Lost updates: 2579530

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceLab>dotnet run --no-build
Expected: 4000000
Actual: 1704134
Lost updates: 2295866
```

Discussion Questions

1. Did the actual value ever equal the expected value? Were any two runs identical? Why is the result non-deterministic even though the source code never changes?

No. The actual value was never 4,000,000; it ranged from 1,330,044 to 1,862,342, and no two runs matched. The OS schedules the threads differently every run, and counter++ isn't atomic, so the result depends on how the threads happen to interleave.

2. Using the load / add / store breakdown of counter++ from Section 3, explain how one increment can be lost when two threads interleave.

counter++ is load → add → store. If Thread A and Thread B both load 100, both add 1 and both store 101. Two increments happened, but the counter only went up by one, so one update is lost.

3. Suppose IncrementsPerThread were reduced to 100. Would you expect the race to disappear, or merely to become harder to observe? Explain why "the output looked correct" is not evidence that a program is free of races.

The race wouldn't go away; it would just be hard to see. With 100 increments, each thread finishes before the next one even starts, so the output usually looks correct. A correct output only means the bad interleaving didn't happen in that run, not that it can't happen.

Section 3: Task 2 Tracing the Race and Interpreting the Report

Source file: RaceTrace/Program.cs

```
using System;
using System.Threading;

class Program
{
    const int NumThreads = 4;
    const int IncrementsPerThread = 250_000;

    static int counter = 0;
    static readonly object gate = new object();
    static bool useLock = false;
    static int[][] seenLog = new int[NumThreads][];

    static void Worker(int id)
    {
        int[] log = seenLog[id];
        for (int i = 0; i < IncrementsPerThread; i++)
        {
            if (useLock)
            {
                lock (gate)
                {
                    int seen = counter;
                    log[i] = seen;
                    counter = seen + 1;
                }
            }
            else
            {
                int seen = counter;
                log[i] = seen;
                counter = seen + 1;
            }
        }
    }

    static void Main(string[] args)
    {
        useLock = args.Length > 0 && args[0] == "--lock";
        int total = NumThreads * IncrementsPerThread;
        Thread[] threads = new Thread[NumThreads];

        for (int t = 0; t < NumThreads; t++)
        {
            seenLog[t] = new int[IncrementsPerThread];
            int id = t;
            threads[t] = new Thread(() => Worker(id));
            threads[t].Start();
        }

        foreach (Thread th in threads) th.Join();

        int[] readCount = new int[total + 1];
        for (int t = 0; t < NumThreads; t++)
            for (int i = 0; i < IncrementsPerThread; i++)
                readCount[seenLog[t][i]]++;

        int collisions = 0;
        for (int v = 0; v <= total; v++)
            if (readCount[v] > 1) collisions += readCount[v] - 1;

        string mode = useLock ? "with lock" : "no synchronization";
    }
}
```

```

Console.WriteLine($"Mode: {mode}");
Console.WriteLine($"Total increments: {total}");
Console.WriteLine($"Final counter: {counter}");
Console.WriteLine($"Lost updates: {total - counter}");
Console.WriteLine($"Collisions: {collisions}");

Console.WriteLine();
Console.WriteLine("First 5 colliding values:");
int printed = 0;
for (int v = 0; v <= total && printed < 5; v++)
{
    if (readCount[v] > 1)
    {
        Console.WriteLine($"Value {v} was loaded {readCount[v]} times, by:");
        for (int t = 0; t < NumThreads; t++)
        {
            for (int i = 0; i < IncrementsPerThread; i++)
            {
                if (seenLog[t][i] == v)
                {
                    Console.WriteLine($" Thread {t}, loop index {i}");
                }
            }
        }
        printed++;
    }
}

if (printed == 0)
{
    Console.WriteLine(" (no collisions found in this run)");
}
}
}

```

Results (3 runs, no synchronization)

Item	Run 1	Run 2	Run 3
Total increments	1,000,000	1,000,000	1,000,000
Final counter value	358,704	440,412	339,259
Lost updates (total – final)	641,296	559,588	660,741
Collisions	641,296	559,588	660,741
First colliding value	2,026	5,352	3,050
Threads that loaded that value	Thread 0 (index 2026), Thread 1 (index 0)	Thread 0 (index 5352), Thread 1 (index 0)	Thread 0 (index 3050), Thread 1 (index 0)

```

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceTrace>dotnet run
Mode: no synchronization
Total increments: 1000000
Final counter: 358704
Lost updates: 641296
Collisions: 641296

```

```

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceTrace>dotnet run
Mode: no synchronization
Total increments: 1000000
Final counter: 440412
Lost updates: 559588
Collisions: 559588

```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC Repo\Lab4_PDC\RaceTrace>dotnet run
Mode: no synchronization
Total increments: 1000000
Final counter: 339259
Lost updates: 660741
Collisions: 660741
```

Interleaving write-up for one collision (choose one from your output and model it on the Section 3 table):

Collision chosen: Run 1, value 2026, loaded by Thread 0 (loop index 2026) and Thread 1 (loop index 0).

Thread 0 started first and ran alone for its first 2,026 iterations, bringing the counter to 2026. Thread 1 then started and, at the same moment Thread 0 was in iteration 2026, both threads read the same value:

Step	Thread 0 (loop index 2026)	Thread 1 (loop index 0)	counter (shared)
1	load → seen = 2026		2026
2		load → seen = 2026	2026
3	add → 2027		2026
4		add → 2027	2026
5	store 2027		2027
6		store 2027 (overwrites)	2027 (should be 2028)

Both threads loaded 2026 and both stored 2027, so two increments raised the counter by only one: one update was lost. The next four collisions in the same run (values 2027–2030, Thread 0 indices 2027–2030 vs. Thread 1 indices 1–4) show the two threads running in lock-step for several iterations, each time reading the same value before the other's store became visible.

Discussion Questions

1. What does a collision mean in terms of the load / add / store steps? Describe, using one real collision from your output, how two threads ended up loading the same value.

A collision means two threads loaded the same value: the second thread loaded before the first thread's store was visible. In run 1, Thread 0 (index 2026) and Thread 1 (index 0) both loaded 2026 and both stored 2027.

Example from run 1: value 2026 was loaded by Thread 0 at loop index 2026 and by Thread 1 at loop index 0. Thread 0 loaded 2026; Thread 1, which had just started, loaded 2026 as well before Thread 0 stored 2027. Both then stored 2027, so one increment vanished.

2. Is the number of lost updates equal to the number of collisions in your results? Compare the two and explain any difference (hint: consider what a thread with a stale value does when it stores).

Yes, they were exactly equal in all 3 runs (641,296 / 559,588 / 660,741). Every duplicate load produces a value someone else already stored, so each collision equals one lost update. A thread with a stale value can also store late and push the counter backwards, so values get loaded again (e.g. value 3052 was loaded 3 times in run 3). That adds to both numbers equally.

3. Suppose a run of this detector reported zero collisions. Would that prove the program is race-free? Explain, and contrast a tool that only sees races that actually happened in a run with one (such as ThreadSanitizer for C/C++) that analyzes whether accesses are ordered by synchronization.

No. This detector only catches races that actually happened in that run. ThreadSanitizer checks whether accesses are ordered by synchronization, so it can report the race even in a run where the result came out correct.

4. Why was it important that each thread wrote to its own private log rather than all threads appending to one shared list?

A shared list would itself be shared state with its own race (lost or corrupted entries). Putting a lock on it would change the timing and hide the race. Private per-thread logs need no sharing or locking.

Section 4: Task 3 Fixing the Race with a Lock (Critical Section)

Source file: MutexFix/Program.cs

```
using System;
using System.Threading;

class Program
{
    const int NumThreads = 4;
    const int IncrementsPerThread = 1_000_000;

    static long counter = 0;
    static readonly object counterLock = new object();

    static void Worker()
    {
        for (int i = 0; i < IncrementsPerThread; i++)
        {
            lock (counterLock)
            {
                counter++;
            }
        }
    }

    static void Main()
    {
        Thread[] threads = new Thread[NumThreads];

        for (int i = 0; i < NumThreads; i++)
        {
            threads[i] = new Thread(Worker);
            threads[i].Start();
        }

        for (int i = 0; i < NumThreads; i++)
        {
            threads[i].Join();
        }

        long expected = (long)NumThreads * IncrementsPerThread;
        Console.WriteLine($"Expected: {expected}");
        Console.WriteLine($"Actual: {counter}");
        Console.WriteLine($"Lost updates: {expected - counter}");
        Console.WriteLine($"Correct: {counter == expected}");
    }
}
```

Results (10 runs)

Run	Expected Value	Actual Value	Correct? (Y/N)
1	4,000,000	4,000,000	Y
2	4,000,000	4,000,000	Y
3	4,000,000	4,000,000	Y
4	4,000,000	4,000,000	Y
5	4,000,000	4,000,000	Y
6	4,000,000	4,000,000	Y
7	4,000,000	4,000,000	Y
8	4,000,000	4,000,000	Y
9	4,000,000	4,000,000	Y
10	4,000,000	4,000,000	Y

```

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\MutexFix>dotnet run --no-build
Expected: 4000000
Actual: 4000000
Lost updates: 0
Correct: True

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\MutexFix>dotnet run --no-build
Expected: 4000000
Actual: 4000000
Lost updates: 0
Correct: True

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\MutexFix>dotnet run --no-build
Expected: 4000000
Actual: 4000000
Lost updates: 0
Correct: True

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\MutexFix>dotnet run --no-build
Expected: 4000000
Actual: 4000000
Lost updates: 0
Correct: True

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\MutexFix>dotnet run --no-build
Expected: 4000000
Actual: 4000000
Lost updates: 0
Correct: True

```

```

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\MutexFix>dotnet run --no-build
Expected: 4000000
Actual: 4000000
Lost updates: 0
Correct: True

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\MutexFix>dotnet run --no-build
Expected: 4000000
Actual: 4000000
Lost updates: 0
Correct: True

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\MutexFix>dotnet run --no-build
Expected: 4000000
Actual: 4000000
Lost updates: 0
Correct: True

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\MutexFix>dotnet run --no-build
Expected: 4000000
Actual: 4000000
Lost updates: 0
Correct: True

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\MutexFix>dotnet run --no-build
Expected: 4000000
Actual: 4000000
Lost updates: 0
Correct: True

```

RaceTrace --lock Results (3 runs)

Run	Total Increments	Final Counter	Collisions
1	1,000,000	1,000,000	0
2	1,000,000	1,000,000	0
3	1,000,000	1,000,000	0

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceTrace>dotnet run -- --lock
Mode: with lock
Total increments: 1000000
Final counter: 1000000
Lost updates: 0
Collisions: 0
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceTrace>dotnet run -- --lock
Mode: with lock
Total increments: 1000000
Final counter: 1000000
Lost updates: 0
Collisions: 0
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\RaceTrace>dotnet run -- --lock
Mode: with lock
Total increments: 1000000
Final counter: 1000000
Lost updates: 0
Collisions: 0
```

Discussion Questions

1. Was the actual value equal to the expected value in all 10 runs? Why is this on its own not conclusive proof that the race is fixed, and what additional evidence did the --lock trace provide? Is even that evidence a proof?

Yes, all 10 runs gave 4,000,000. That alone isn't proof, since racy code can also look correct. The --lock trace showed 0 collisions, meaning no two threads ever loaded the same value. Even that is still testing, not proof; the real argument is that every access to `counter` is inside the same lock.

2. Main() reads counter after the joins without taking the lock. Why is that safe here? (Hint: what does Thread.Join() guarantee about the finished threads' writes?)

`Join()` waits until the thread has finished and guarantees its writes are visible to `Main()`. After all the joins, no other thread is running, so there's nothing to race with.

3. What would happen to correctness if only three of the four threads used the lock and one still executed a bare counter++? What if each thread locked a different object?

If one thread uses a bare `counter++`, updates can be lost again, because a lock only protects code that actually takes it. If each thread locks a different object, there's no mutual exclusion at all, so it's as racy as Task 1.

4. The lock is placed around each single increment rather than around the whole loop. What is the trade-off between the two placements in terms of correctness, parallelism, and lock overhead?

Locking each increment is correct but pays for 4 million lock/unlock pairs with heavy contention. Locking the whole loop needs only 4 lock/unlock pairs, but the threads run one after another with no parallelism. The trade-off is lock overhead versus how much of the work can overlap.

Section 5: Task 4 Atomic Increment vs. Critical Section

Source file: CounterBench/Program.cs

```
using System;
using System.Diagnostics;
using System.Threading;

class Program
{
    const int NumThreads = 4;
    const int IncrementsPerThread = 1_000_000;

    static long counter = 0;
    static readonly object counterLock = new object();

    static void LockWorker()
    {
        for (int i = 0; i < IncrementsPerThread; i++)
        {
            lock (counterLock) { counter++; }
        }
    }

    static void InterlockedWorker()
    {
        for (int i = 0; i < IncrementsPerThread; i++)
        {
            Interlocked.Increment(ref counter);
        }
    }

    static double RunVersion(string name, ThreadStart worker)
    {
        counter = 0;
        Thread[] threads = new Thread[NumThreads];

        Stopwatch sw = Stopwatch.StartNew();

        for (int i = 0; i < NumThreads; i++)
        {
            threads[i] = new Thread(worker);
            threads[i].Start();
        }

        for (int i = 0; i < NumThreads; i++)
        {
            threads[i].Join();
        }

        sw.Stop();

        long expected = (long)NumThreads * IncrementsPerThread;
        double ms = sw.Elapsed.TotalMilliseconds;

        Console.WriteLine($"[{name}] expected={expected} actual={counter} " +
            $"correct={counter == expected} time={ms:F2} ms");

        return ms;
    }

    static void Main()
    {
        RunVersion("warm-up lock", LockWorker);
        RunVersion("warm-up Interlocked", InterlockedWorker);
    }
}
```

```

    RunVersion("lock", LockWorker);
    RunVersion("Interlocked.Increment", InterlockedWorker);
}
}

```

Results (5 runs, Release configuration)

Run	lock Time (ms)	Interlocked.Increment Time (ms)
1	217.13	96.89
2	153.20	89.80
3	168.19	92.97
4	176.45	84.04
5	165.65	89.86
Average	176.12	90.71

Correct in all runs? (Y/N): Y (every lock and Interlocked run printed correct=True)

Ratio (average lock time / average Interlocked time): $176.12 / 90.71 \approx 1.94\times$

Number of threads and increments per thread used: 4 threads $\times$ 1,000,000 increments each

```

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\CounterBench>dotnet run --no-build -c Release
[warm-up lock] expected=4000000 actual=4000000 correct=True time=164.73 ms
[warm-up Interlocked] expected=4000000 actual=4000000 correct=True time=98.52 ms
[lock] expected=4000000 actual=4000000 correct=True time=217.13 ms
[Interlocked.Increment] expected=4000000 actual=4000000 correct=True time=96.89 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\CounterBench>dotnet run --no-build -c Release
[warm-up lock] expected=4000000 actual=4000000 correct=True time=152.08 ms
[warm-up Interlocked] expected=4000000 actual=4000000 correct=True time=88.97 ms
[lock] expected=4000000 actual=4000000 correct=True time=153.20 ms
[Interlocked.Increment] expected=4000000 actual=4000000 correct=True time=89.80 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\CounterBench>dotnet run --no-build -c Release
[warm-up lock] expected=4000000 actual=4000000 correct=True time=176.32 ms
[warm-up Interlocked] expected=4000000 actual=4000000 correct=True time=105.01 ms
[lock] expected=4000000 actual=4000000 correct=True time=168.19 ms
[Interlocked.Increment] expected=4000000 actual=4000000 correct=True time=92.97 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\CounterBench>dotnet run --no-build -c Release
[warm-up lock] expected=4000000 actual=4000000 correct=True time=173.14 ms
[warm-up Interlocked] expected=4000000 actual=4000000 correct=True time=93.08 ms
[lock] expected=4000000 actual=4000000 correct=True time=176.45 ms
[Interlocked.Increment] expected=4000000 actual=4000000 correct=True time=84.04 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab4_PDC\CounterBench>dotnet run --no-build -c Release
[warm-up lock] expected=4000000 actual=4000000 correct=True time=211.96 ms
[warm-up Interlocked] expected=4000000 actual=4000000 correct=True time=95.82 ms
[lock] expected=4000000 actual=4000000 correct=True time=165.65 ms
[Interlocked.Increment] expected=4000000 actual=4000000 correct=True time=89.86 ms

```

Discussion Questions

1. Did both versions produce the correct total in every run? Compare this with the unsynchronized program in Task 1.

Yes, both printed correct=True in all 5 runs. In Task 1, the unsynchronized version lost about 2.1–2.7 million updates every run.

2. Which version was faster on your machine, and by roughly what factor? Give two reasons why an atomic increment tends to be cheaper than a lock/unlock pair around the same increment.

Interlocked was faster: an average of 90.71 ms versus 176.12 ms for lock, about $1.94\times$. There are two reasons:

- It's a single atomic CPU instruction, while a lock needs acquire, increment and release.
- A contended lock makes threads spin or block, with context switches; Interlocked never waits for a lock holder.

3. Even the Interlocked version is far slower than a single thread incrementing an unshared variable. What causes this, given that all threads write the same memory location? How does the one-slot-per-thread approach from Lab 03, Task 2 avoid it?

All threads write the same memory location, so its cache line bounces between cores on every increment. The Lab 03 one-slot-per-thread approach gives each thread its own counter and adds them at the end, so nothing is shared.

4. An atomic increment protects one variable. Give an example of an update that Interlocked cannot make safe but a lock can (for instance, an update that must change two related fields together), and explain why.

Example: a bank transfer (`a -= x; b += x;`). Interlocked can make each operation atomic, but another thread can see the in-between state where the money has left A but not reached B. A lock makes both updates happen as one step.

5. How would you expect the ratio of lock time to Interlocked time to change with 1 thread, and with more threads than cores? Justify your answer, and if time permits, test it by changing NumThreads to 1 and to 8.

With 1 thread there's no contention, so both get faster and the ratio shrinks to about 1–2×. With more threads than cores (e.g. 16 threads on your 8 cores), a thread can be paused while holding the lock and block all the others, so the ratio should grow. These are expectations only; you didn't run those cases.

Section 6: Github Repository

All source code for this lab (RaceLab, RaceTrace, MutexFix and CounterBench) is pushed to the lab4 branch of my repository:

https://github.com/fahad1084/-PDC_Labs/tree/lab4